چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۶

ضروریات Jetpack — ViewModel، SavedStateHandle، StateFlow، Room، DataStore و Paging



اسماعیل مرادی

@esmaeeil-moradi700

چرا جزئیات درونی Jetpack در ۲۰۲۶ مهم است؟

«با ViewModel و Room کار کرده‌ام» دیگر کافی نیست. مصاحبه‌گرها می‌پرسند **درونشان چطور کار می‌کند** و **چه کاری را انجام نمی‌دهند**:

- مقایسهٔ ViewModel با `SavedStateHandle`: چه چیزی از چرخش صفحه عبور می‌کند و چه چیزی از process death
- LiveData منسوخ نشده — اما `StateFlow` انتخاب پیش‌فرض برای کد جدید و چندسکویی است
- relationها، migrationها و transactionهای Room: جایی که دادهٔ کاربر در production گم می‌شود
- DataStore جای SharedPreferences را گرفته؛ Paging 3 لیست‌های بزرگ را مدیریت می‌کند
- ViewModel، Room، DataStore و هستهٔ Paging حالا در کد مشترک KMP اجرا می‌شوند

سطح	سؤال‌ها
جونیور	درون ViewModel، مقایسهٔ LiveData و StateFlow، مبانی Room
مید	SavedStateHandle، factoryها و CreationExtras، relationها، DataStore
سینیور	migrationها، Room زیر بار، Paging 3
آرکیتکت	Room در KMP، migration در production، یک دیتابیس یا چند دیتابیس



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: چه چیزی از چه چیزی عبور می‌کند

	Rotation	Process death	Swiped away
remember { }	no	no	no
rememberSaveable	yes	yes	no
ViewModel	yes	no	no
SavedStateHandle	yes	yes	no
Room · DataStore	yes	yes	yes

- **ViewModel** — یک cache در حافظه برای یک صفحه
- **SavedStateHandle** — کلیدهای کوچکی که از process death به ابتکار سیستم عبور می‌کنند
- **Room و DataStore** — حقیقت روی دیسک؛ بعد از process death صفحه از *کلیدها به‌علاوهٔ دیسک* دوباره ساخته می‌شود

سؤال پشت همهٔ جواب‌های JETPACK

چه چیزی باید از چه چیزی عبور کند — و مالکش کیست؟ ارزان‌ترین نگه‌دارنده‌ای را انتخاب کنید که از همان چیزی که لازم دارید عبور می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

درون ViewModel، مقایسهٔ LiveData و StateFlow، مبانی Room



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. ViewModel چطور از چرخش صفحه عبور می‌کند؟

- `by viewModels()` یا `viewModel()` نمونه را از `ViewModelProvider` می‌خواهد. نمونه در **`ViewModelStore`** مالک نگه داشته می‌شود؛ یک map که کلیدش اسم کلاس است. فقط اگر نباشد ساخته می‌شود.
- هنگام چرخش صفحه، `ComponentActivity` این store را به‌عنوان یک **non-configuration instance** به نمونهٔ جدید می‌دهد؛ پس همان شیء ViewModel برمی‌گردد.
- وقتی Activity برای همیشه بسته شود، store پاک می‌شود.

Rotation	finish() / Back
old Activity destroyed, store handed over new Activity: same store, same ViewModel instance	ON_DESTROY and not isChangingConfigurations → viewModelStore.clear() → onCleared(), scope cancelled

سؤال تکمیلی

«*ViewModelهای یک Fragment کجا نگه داشته می‌شوند؟*» — در یک `FragmentManagerViewModel` که خودش یک ViewModel در store مربوط به Activity است. برای همین آن‌ها هم از چرخش صفحه عبور می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. مالک‌ها و پاک‌سازی

مالک	ViewModelهایش کِی پاک می‌شوند
`ComponentActivity`	وقتی finish شود — نه با چرخش صفحه
`Fragment`	وقتی برای همیشه حذف شود — نه وقتی در back stack است
`NavBackStackEntry`	وقتی آن destination از back stack بیرون برود
`viewModel()` در Compose	از نزدیک‌ترین `LocalViewModelStoreOwner` استفاده می‌کند

```
class PlayerViewModel(player: AudioPlayer) : ViewModel() {
    init {
        addCloseable(player) // closed when the VM is cleared
    }
}
```

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت: «*ViewModel یک کلاس lifecycle-aware است.*» — نه؛ ViewModel فقط **به یک lifecycle محدود** است و هیچ‌وقت آن را observe نمی‌کند. فقط در یک `ViewModelStore` می‌ماند تا وقتی مالک پاکش کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. مقایسهٔ LiveData و StateFlow

	LiveData	StateFlow
lifecycle	داخلی است: فقط به observerهای در حالت STARTED یا RESUMED می‌رساند	ندارد — با `repeatOnLifecycle` یا `collectAsStateWithLifecycle` collect کنید
مقدار اولیه	اختیاری (می‌تواند خالی باشد)	الزامی
تکرار همان مقدار	`setValue` همیشه خبر می‌دهد	رد می‌شود (مقایسه با `equals`)
تردها	`setValue` فقط روی ترد اصلی؛ `postValue` از هر تردی	`value` و `update {}` از هر تردی
operatorها	`map`، `switchMap`، `distinctUntilChanged` و `MediatorLiveData`	همهٔ operatorهای Flow
پلتفرم	فقط اندروید	کاتلین خالص: KMP و تست ساده روی JVM

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت: «*LiveData متد `distinctUntilChanged()` ندارد و به transformer سفارشی نیاز دارید.*» — اشتباه است: `distinctUntilChanged()`، `map` و `switchMap` همراه `lifecycle-livedata` عرضه می‌شوند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. collect امن یک StateFlow

```
// Views: collect only while the view is at least STARTED
val owner = viewLifecycleOwner
owner.lifecycleScope.launch {
    owner.repeatOnLifecycle(Lifecycle.State.STARTED) {
        viewModel.uiState.collect(::render)
    }
}

// Compose
val state by viewModel.uiState.collectAsStateWithLifecycle()

// Bridges while migrating
val flow: Flow<User> = userLiveData.asFlow()
val live: LiveData<User> = userFlow.asLiveData()
```

تلهٔ مصاحبه

«`postValue` همهٔ مقدارها را می‌رساند.» — نه. اگر قبل از اجرای ترد اصلی چند بار post شود، فقط **آخرین** مقدار ارسال می‌شود. برای state مشکلی نیست، برای event اشتباه است.

تلهٔ مصاحبه

«*LiveData منسوخ شده.*» — نشده. پایدار است و پشتیبانی می‌شود؛ فقط StateFlow برای کد جدید، Compose و KMP پیش‌فرض بهتری است. اما `launchWhenStarted` **واقعاً** deprecated است — به‌جایش از `repeatOnLifecycle` استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. مبانی Room: entity، DAO و database

```kotlin
@Entity(tableName = "notes")
data class NoteEntity(
    @PrimaryKey val id: String,
    val title: String,
    val updatedAt: Long,
)

@Dao
interface NoteDao {
    @Query("SELECT * FROM notes ORDER BY updatedAt DESC")
    fun observeAll(): Flow<List<NoteEntity>>

    @Upsert suspend fun upsert(note: NoteEntity)
}

@Database(entities = [NoteEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun noteDao(): NoteDao
}
```

در زمان کامپایل بررسی می‌شود SQL (کامپایلر Room روی KSP اجرا می‌شود). هر query از نوع `Flow` با هر تغییر در جدول‌هایش دوباره emit می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. قواعدی که مصاحبه‌گر چک می‌کند

- تابع‌های `suspend` در DAO **main-safe** هستند: Room آن‌ها را روی executorهای خودش اجرا می‌کند.
- query روی ترد اصلی `IllegalStateException` می‌دهد — مگر اینکه خودتان این چک را خاموش کنید، که نباید بکنید.
- در هر process فقط یک نمونه از `RoomDatabase` — بگذارید DI مالکش باشد (روز ۷).
- نوع ستون‌های داخلی: primitiveها، `String`، `ByteArray` و enumها. بقیه به `@TypeConverter` نیاز دارند.

تلهٔ مصاحبه

پیچیدن فراخوانی‌های DAO در `withContext(Dispatchers.IO)`. اضافی است — DAO از نوع `suspend` خودش main-safe است؛ فقط یک جابه‌جایی ترد اضافه می‌کنید.

تلهٔ مصاحبه

در نسخهٔ پارسال: `exportSchema = false` به‌علاوهٔ یک singleton دست‌ساز با double-checked locking. export schema کردن را روشن نگه دارید (پیش‌فرض همین است) و از Room Gradle plugin استفاده کنید — `room { schemaDirectory("$projectDir/schemas") }` — تا migrationها قابل تولید و تست باشند. فایل‌های JSON را commit کنید.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

SavedStateHandle، factoryها و CreationExtras، relationها، DataStore



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. SavedStateHandle چطور کار می‌کند

```
class SearchViewModel(
    private val handle: SavedStateHandle,
    private val repo: SearchRepository,
) : ViewModel() {
    // Restored after system-initiated process death
    val query: StateFlow<String> =
        handle.getStateFlow(QUERY, "")

    val results: StateFlow<List<Repo>> = query
        .flatMapLatest { repo.search(it) }
        .stateIn(
            viewModelScope,
            SharingStarted.WhileSubscribed(5_000),
            emptyList(),
        )

    fun onQueryChange(q: String) { handle[QUERY] = q }
}
```

- از روی `CreationExtras` ساخته می‌شود: `SavedStateRegistry` مالک، به‌علاوهٔ **آرگومان‌های پیش‌فرض** (extraهای intent، آرگومان‌های Fragment، route در navigation)
- یک `SavedStateProvider` ثبت می‌کند: وقتی مالک state را ذخیره می‌کند، map مربوط به handle داخل `Bundle` مالک نوشته می‌شود



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. محدودیت‌هایی که باید بدانید

- مقدارها باید در `Bundle` جا شوند: primitiveها، `String`، `Parcelable` و... **شناسه‌ها، عبارت جست‌وجو، تب انتخاب‌شده** را ذخیره کنید — نه لیست داده‌ها.
- Bundle از طریق Binder منتقل می‌شود. بافر تراکنش هر process فقط ۱ مگابایت است و بین همهٔ تراکنش‌های در جریان مشترک است؛ state زیاد یعنی `TransactionTooLargeException`.
- وقتی کاربر task را از Recents ببندد یا اپ را force stop کند، از بین می‌رود.

تلهٔ مصاحبه

«*SavedStateHandle روی دیسک می‌نویسد.*» — نه. این همان `Bundle` مربوط به instance state است که سیستم نگه می‌دارد. داده‌ای که باید بماند جایش در Room یا DataStore است.

تلهٔ مصاحبه

گذاشتن کل `UiState` در `SavedStateHandle` «برای اطمینان». خطر `TransactionTooLargeException` و دادهٔ کهنه. کلیدها را ذخیره کنید؛ داده را از repository بخوانید (روز ۳).

سؤال تکمیلی

«`rememberSaveable` یا *`SavedStateHandle`*؟» — `rememberSaveable` برای state عناصر UI مثل باز بودن یا اسکرول؛ `SavedStateHandle` برای stateی که منطق صفحه لازم دارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. ساختن ViewModel: factoryها و CreationExtras

```
class DetailViewModel(
    private val repo: DetailRepository,
    handle: SavedStateHandle,
) : ViewModel() {
    companion object {
        val Factory = viewModelFactory {
            initializer {
                val app = this[APPLICATION_KEY] as MyApp
                DetailViewModel(
                    repo = app.container.detailRepository,
                    handle = createSavedStateHandle(),
                )
            }
        }
    }
}
// Fragment: by viewModels { DetailViewModel.Factory }
// Compose:  viewModel(factory = DetailViewModel.Factory)
```

`CreationExtras` هر چیزی را که factory لازم دارد حمل می‌کند: `APPLICATION_KEY`، `VIEW_MODEL_STORE_OWNER_KEY`، `SAVED_STATE_REGISTRY_OWNER_KEY` و `DEFAULT_ARGS_KEY`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. داخل `get()` چه اتفاقی می‌افتد

۱. `ViewModelProvider(owner, factory)[DetailViewModel::class]` از روی اسم کلاس، یا `key` شما، یک کلید می‌سازد.

۲. اگر در `ViewModelStore` باشد، همان برمی‌گردد و **factory صدا زده نمی‌شود**.

۳. اگر نباشد، `factory.create(modelClass, extras)` صدا زده می‌شود و نتیجه زیر همان کلید ذخیره می‌شود.

در Hilt، `@HiltViewModel` و در Koin، `viewModelOf` این factory را برایتان تولید می‌کنند (روز ۷).

تلهٔ مصاحبه

نوشتن `val vm = DetailViewModel(repo)` داخل `onCreate()`. این نمونه در هیچ storeی نیست: با هر چرخش صفحه نمونهٔ جدید ساخته می‌شود، `onCleared()` هیچ‌وقت اجرا نمی‌شود و `viewModelScope` هیچ‌وقت لغو نمی‌شود.

تلهٔ مصاحبه

پاس دادن یک `itemId` جدید به factory بعد از چرخش صفحه. factory یک بار اجرا شده و همان نمونهٔ قبلی را پس می‌گیرید. آرگومان‌های هر صفحه جایشان در `SavedStateHandle` است (آرگومان‌های navigation) — یا از یک `key` متفاوت استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. relationها در Room: یک‌به‌چند و چندبه‌چند

```
data class UserWithPlaylists(
    @Embedded val user: User,
    @Relation(parentColumn = "userId", entityColumn = "ownerId")
    val playlists: List<Playlist>,
)

@Entity(primaryKeys = ["playlistId", "songId"])
data class PlaylistSongCrossRef(
    val playlistId: Long,
    val songId: Long,
)

data class PlaylistWithSongs(
    @Embedded val playlist: Playlist,
    @Relation(
        parentColumn = "playlistId",
        entityColumn = "songId",
        associateBy = Junction(PlaylistSongCrossRef::class),
    )
    val songs: List<Song>,
)
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. query درست روی relationها

```
@Dao
interface MusicDao {
    @Transaction
    @Query("SELECT * FROM User")
    fun usersWithPlaylists(): Flow<List<UserWithPlaylists>>

    // Alternative: one JOIN returning a multimap
    @Query(
        "SELECT * FROM User " +
            "JOIN Playlist ON User.userId = Playlist.ownerId"
    )
    suspend fun playlistsByUser(): Map<User, List<Playlist>>
}
```

- Room **یک query برای والدها و queryهای دیگری برای فرزندها** اجرا می‌کند. `@Transaction` همه را یک خواندن سازگار می‌کند — اگر نباشد، Room هشدار می‌دهد.

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت: «`@Relation` *فقط یک سطح تودرتو را پشتیبانی می‌کند.*» — اشتباه است. برای تودرتو کردن، `@Relation(entity = Playlist::class, …)` را به کلاسی اشاره دهید که خودش relation دارد، مثلاً `List<PlaylistWithSongs>`.

تلهٔ مصاحبه

`@Relation` کلید خارجی **نیست**. برای یکپارچگی داده روی جدول فرزند `ForeignKey(onDelete = CASCADE)` بگذارید — و آن ستون را index کنید، وگرنه Room هشدار می‌دهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. مقایسهٔ DataStore، SharedPreferences و Room

	SharedPreferences	DataStore	Room
API	خواندن هم‌زمان؛ `apply()` و `commit()`	`Flow` و نوشتن با `suspend`	DAO و SQL
ریسک ترد اصلی	بله: بارگذاری اول، `commit()` و `apply()` های معلق که در `onStop` flush می‌شوند	خیر	خیر
خطاها	بی‌صدا	`IOException` داخل Flow	exception
سازگاری	بدون transaction	خواندن، تغییر و نوشتن اتمیک	transaction
مناسب برای	فقط کد قدیمی	تنظیمات کوچک، flagها	داده‌های رابطه‌ای یا حجیم

- **Preferences DataStore:** کلیدهای تایپ‌دار، بدون schema
- **DataStore تایپ‌دار:** با Proto، یا JSON از طریق kotlinx.serialization؛ یک data class کامل همراه با یک `Serializer`
- دیتابیس نیست: query ندارد و هر ویرایش کل فایل را بازنویسی می‌کند
- DataStore چندسکویی است — همان کد در `commonMain`



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. DataStore در کد

```
val Context.settings by preferencesDataStore(
    name = "settings",
    produceMigrations = { ctx ->
        listOf(SharedPreferencesMigration(ctx, "prefs"))
    },
)
private val DARK_MODE = booleanPreferencesKey("dark_mode")

class SettingsRepository(
    private val store: DataStore<Preferences>,
) {
    val darkMode: Flow<Boolean> = store.data
        .catch { e ->
            if (e !is IOException) throw e
            emit(emptyPreferences())
        }
        .map { it[DARK_MODE] ?: false }

    suspend fun setDarkMode(on: Boolean) {
        store.edit { it[DARK_MODE] = on }
    }
}
```

تلهٔ مصاحبه

دو نمونه DataStore برای **یک فایل** در یک process یعنی `IllegalStateException`. فقط یک بار تعریفش کنید: یک delegate در سطح فایل یا یک singleton در DI.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

migrationها، Room زیر بار، Paging 3



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۸. migration: اول خودکار

```
@Database(
    entities = [User::class],
    version = 3,
    autoMigrations = [
        AutoMigration(from = 1, to = 2),
        AutoMigration(from = 2, to = 3, spec = AppDb.V3::class),
    ],
)
abstract class AppDb : RoomDatabase() {
    @RenameColumn(
        tableName = "User",
        fromColumnName = "name",
        toColumnName = "fullName",
    )
    class V3 : AutoMigrationSpec
}
```

- AutoMigration تفاوت **فایل‌های JSON مربوط به schema** در دو نسخه را درمی‌آورد — export نباشد، auto-migration هم نیست.
- تغییرهای مبهم، مثل تغییر نام یا حذف ستون، به یک spec نیاز دارند؛ وگرنه build شکست می‌خورد. بقیهٔ موارد: یک `Migration` دستی.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. migration دستی — و تست آن

```
val MIGRATION_3_4 = object : Migration(3, 4) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("ALTER TABLE User ADD COLUMN email TEXT")
    }
}

@get:Rule val helper = MigrationTestHelper(
    InstrumentationRegistry.getInstrumentation(),
    AppDb::class.java,
)

@Test fun migrate3To4() {
    helper.createDatabase(DB_NAME, 3).close()
    helper.runMigrationsAndValidate(
        DB_NAME, 4, true, MIGRATION_3_4,
    )
}
```

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت: *«بدون migration، Room دیتابیس را پاک می‌کند و از نو می‌سازد.»* — نه: Room هنگام باز کردن `IllegalStateException` می‌دهد. داده فقط وقتی پاک می‌شود که خودتان یکی از متدهای `fallbackToDestructiveMigration` را فعال کرده باشید.

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت: *«کنار migration از ۱ به ۲ و از ۲ به ۳، باید از ۱ به ۳ را هم بنویسید.»* — لازم نیست؛ Room migrationها را زنجیره‌وار اجرا می‌کند. migration مستقیم از ۱ به ۳ فقط یک میان‌بر اختیاری است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. Room زیر بار: index و transaction

```
@Entity(
    indices = [
        Index("ownerId"),
        Index(value = ["email"], unique = true),
    ],
)
data class Account(
    @PrimaryKey val id: Long,
    val ownerId: Long,
    val email: String,
)

// One transaction: one commit instead of thousands
suspend fun replaceAll(items: List<ItemEntity>) =
    db.withTransaction {
        itemDao.deleteAll()
        itemDao.insertAll(items)
    }
```

- ستون‌هایی را که در `WHERE`، `JOIN`، `ORDER BY` و کلیدهای خارجی به کار می‌روند index کنید. هر index فضا می‌گیرد و نوشتن را کند می‌کند.
- نوشتن‌ها را دسته‌ای و در یک transaction انجام دهید: خیلی سریع‌تر است، و خواننده‌ها هیچ‌وقت نیمی از دسته را نمی‌بینند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. queryهای observable و قفل‌ها

- یک query از نوع `Flow` وقتی **هر** جدولی که می‌خواند نوشته شود دوباره اجرا می‌شود — حتی برای ردیف‌هایی بیرون از نتیجه‌اش. Room نتیجهٔ جدید را حتی اگر برابر باشد emit می‌کند؛ پس `distinctUntilChanged()` اضافه کنید.
- یک `SELECT *` بزرگ را observe نکنید: فقط ستون‌های لازم را انتخاب کنید، یا صفحه‌بندی‌اش کنید (سؤال ۱۰).
- Room روی بیشتر دستگاه‌ها حالت **WAL** یعنی write-ahead logging را فعال می‌کند: خواننده‌ها نویسنده را بلاک نمی‌کنند — اما هنوز در هر لحظه فقط **یک نویسنده** وجود دارد. transactionها را کوتاه نگه دارید.

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت: «*partial update فقط ستون‌های تغییرکرده را قفل می‌کند.*» — SQLite قفل ستونی یا ردیفی ندارد: هر نوشتن **کل دیتابیس** را قفل می‌کند. partial updateها — با `@Update(entity = Account::class)` و یک کلاس جزئی، یا با `UPDATE … SET` — در serialization و I/O صرفه‌جویی می‌کنند، نه در رقابت بر سر قفل.

اندازه بگیرید

ابزار **Database Inspector** در Android Studio روی دستگاه query زنده اجرا می‌کند؛ همان‌جا `EXPLAIN QUERY PLAN` را اجرا کنید تا مطمئن شوید query از index شما استفاده می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. Paging 3: اجزا

```
PagingSource ──► Pager(PagingConfig) ──► Flow<PagingData>
(loads pages)   (in the ViewModel)     .cachedIn(viewModelScope)
                                                  │
            LazyColumn ◄── collectAsLazyPagingItems()
```

```kotlin
// DAO — Room generates the PagingSource (room-paging)
@Query("SELECT * FROM articles ORDER BY publishedAt DESC")
fun pagingSource(): PagingSource<Int, ArticleEntity>

// ViewModel
val articles: Flow<PagingData<Article>> =
    Pager(PagingConfig(pageSize = 30)) { dao.pagingSource() }
        .flow
        .map { page -> page.map { it.toDomain() } }
        .cachedIn(viewModelScope)
```

وقتی جدول تغییر کند، Room آن `PagingSource` را **invalidate** می‌کند؛ `Pager` یک نمونهٔ جدید می‌سازد و از اطراف موقعیت فعلی دوباره بارگذاری می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. Paging در UI

```
val items = viewModel.articles.collectAsLazyPagingItems()

LazyColumn {
    items(
        count = items.itemCount,
        key = items.itemKey { it.id },
    ) { index ->
        val article = items[index]
        if (article != null) ArticleRow(article)
        else ArticlePlaceholder()
    }
}
// Errors: loadState.refresh is LoadState.Error → items.retry()
```

تلهٔ مصاحبه

فراموش کردن `cachedIn(viewModelScope)`. با چرخش صفحه همه چیز از اول بارگذاری می‌شود و collect دوبارهٔ همان `PagingData` خطا می‌دهد. تبدیل‌ها (`map`، `filter`) را **قبل از** `cachedIn` بگذارید.

تلهٔ مصاحبه

در نسخهٔ پارسال `DataSource.Factory` و `PagedList` آمده بود: API قدیمی و deprecated در Paging 2. نسخهٔ ۳ از `PagingSource` و `PagingData` استفاده می‌کند.

سؤال تکمیلی

«*شبکه به‌علاوهٔ دیتابیس؟*» — Room منبع است و یک `RemoteMediator` آن را پر می‌کند (روز ۱۴).



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

Room در KMP، migration در production، یک دیتابیس یا چند دیتابیس



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. Room در Kotlin Multiplatform

```kotlin
// commonMain
@Database(entities = [NoteEntity::class], version = 1)
@ConstructedBy(AppDatabaseConstructor::class)
abstract class AppDatabase : RoomDatabase() {
    abstract fun noteDao(): NoteDao
}

// Room's compiler generates the actual objects
@Suppress("KotlinNoActualForExpect")
expect object AppDatabaseConstructor :
    RoomDatabaseConstructor<AppDatabase> {
    override fun initialize(): AppDatabase
}

fun buildDatabase(b: RoomDatabase.Builder<AppDatabase>) = b
    .setDriver(BundledSQLiteDriver())
    .setQueryCoroutineContext(Dispatchers.IO)
    .build()
```

هر پلتفرم فقط builder را می‌سازد: در اندروید `Room.databaseBuilder<AppDatabase>(context, path)` و در iOS `Room.databaseBuilder<AppDatabase>(name = path)`.



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۱۱. در KMP چه چیزی عوض می‌شود

- DAOها در کد مشترک باید `suspend` باشند یا `Flow` برگردانند — نه `LiveData` بلاک‌کننده، نه DAO
- migrationها و callbackها از `SQLiteConnection` استفاده می‌کنند، نه `SupportSQLiteDatabase` که فقط اندرویدی است
- **SQLite همراه اپ:** همان SQLite مدرن روی همهٔ پلتفرم‌ها، به قیمت یک کتابخانهٔ native برای هر ABI

کتابخانه	در `commonMain`؟
ViewModel و `viewModelScope`	بله — از Lifecycle 2.8
Room	بله — از نسخهٔ 2.7
DataStore	بله
Paging	فقط هسته (`paging-common`)
LiveData و WorkManager	خیر — فقط اندروید

تلهٔ مصاحبه

«کد *Room اندروید را همان‌طور به `commonMain` ببرید.*» — اول باید DAOهای بلاک‌کننده، خروجی‌های `LiveData` و Rx، `SupportSQLite*` در migrationها و converterهایی که از تایپ‌های اندرویدی استفاده می‌کنند کنار بروند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. migration در production: یک استراتژی

تغییر	استراتژی
جدول جدید، ستون nullable یا دارای مقدار پیش‌فرض	`AutoMigration`
تغییر نام یا حذف ستون یا جدول	`AutoMigration` همراه با spec
تغییر نوع، ادغام یا جدا کردن ستون‌ها، اصلاح داده	دستی: ساخت جدول جدید ← کپی و تبدیل داده ← حذف جدول قدیم ← تغییر نام جدول جدید
دیتابیسی که فقط cache است	fallback تخریبی قابل قبول است
کاربر نسخهٔ قدیمی‌تر را نصب می‌کند	به‌طور پیش‌فرض exception می‌دهد — صریح تصمیم بگیرید، مثلاً با `fallbackToDestructiveMigrationOnDowngrade`

- **migrationی را که منتشر شده هرگز ویرایش نکنید** — یک نسخهٔ جدید اضافه کنید.
- در CI از **هر نسخهٔ منتشرشدهٔ schema** تا نسخهٔ فعلی، با دادهٔ واقعی‌نما تست بگیرید.
- هر migration داخل یک transaction اجرا می‌شود و بعد از آن، schema توسط Room **اعتبارسنجی** می‌شود. عدم تطابق هنگام باز کردن دیتابیس crash می‌دهد، نه اینکه بی‌صدا داده را خراب کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. migrationی که داده را تبدیل می‌کند

```
val MIGRATION_7_8 = object : Migration(7, 8) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            "CREATE TABLE Product_new (" +
                "id INTEGER NOT NULL, name TEXT NOT NULL, " +
                "priceCents INTEGER NOT NULL, PRIMARY KEY(id))"
        )
        db.execSQL(
            "INSERT INTO Product_new (id, name, priceCents) " +
                "SELECT id, name, " +
                "CAST(ROUND(price * 100) AS INTEGER) " +
                "FROM Product"
        )
        db.execSQL("DROP TABLE Product")
        db.execSQL("ALTER TABLE Product_new RENAME TO Product")
        db.execSQL(
            "CREATE INDEX index_Product_name ON Product(name)"
        )
    }
}
```

تلهٔ مصاحبه

فراموش کردن ساخت دوبارهٔ indexها بعد از بازسازی جدول؛ نتیجه: اعتبارسنجی schema در Room شکست می‌خورد. دستور `createSql` را دقیقاً از فایل JSON مربوط به schema کپی کنید.



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۱۳. یک دیتابیس یا چند دیتابیس؟

یک دیتابیس	چند دیتابیس
transaction و JOIN روی همهٔ جدول‌ها	transaction یا JOIN بین فایل‌ها ممکن نیست
یک invalidation tracker و یک نویسنده	هر فایل connectionها و نویسندهٔ خودش را دارد
`@Database` همهٔ entityها را فهرست می‌کند؛ یعنی یک ماژول جدول‌های همهٔ فیچرها را می‌بیند	هر فیچر مالک schema و migrationهای خودش است
یک تاریخچهٔ نسخه	برای هر دیتابیس یک تاریخچهٔ نسخه

پیش‌فرض: یک دیتابیس در ماژول `:core:database` ؛ فیچرها repositoryها را می‌بینند، نه خود دیتابیس را. وقتی چرخهٔ عمر فرق دارد، **جدا کنید:**

- یک دیتابیس **مخصوص هر کاربر** که هنگام logout حذف می‌شود
- یک دیتابیس **cache** که می‌شود دورش ریخت
- ماژولی که مال تیم دیگری است یا جداگانه تحویل داده می‌شود



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. یک دیتابیس برای هر کاربر

```
fun openUserDb(context: Context, userId: String): UserDb =
    Room.databaseBuilder(
        context, UserDb::class.java, "user-$userId.db",
    ).build()

fun deleteUserData(context: Context, db: UserDb, id: String) {
    db.close()
    context.deleteDatabase("user-$id.db")
}
```

تلهٔ مصاحبه

نسخهٔ پارسال می‌گفت: «*دیتابیس‌های جدا امنیت را بیشتر می‌کنند.*» — همهٔ فایل‌های دیتابیس در همان sandbox اپ و با همان دسترسی‌ها هستند. جدا کردن، مرز امنیتی نیست؛ رمزنگاری داده در حالت ذخیره هست (روز ۱۴ و روز ۱۹).

جواب سینیور در یک جمله

«حافظه برای صفحه، Bundle برای کلیدها، دیسک برای حقیقت — و هر تغییر schema یک migration است که قبل از اجرا روی گوشی کاربر تستش کرده‌ام.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۶ — خلاصهٔ یک‌صفحه‌ای

- ViewModel در `ViewModelStore` می‌ماند و با `finish()` پاک می‌شود. **محدود به lifecycle** است، نه lifecycle-aware.
- `SavedStateHandle` یعنی کلیدهای کوچک در Bundle؛ از process death عبور می‌کند، نه از بستن اپ در Recents.
- factory برای هر مالک و کلید فقط یک بار اجرا می‌شود؛ هرگز `MyViewModel(repo)` دستی.
- LiveData منسوخ نشده؛ `StateFlow` با `collectAsStateWithLifecycle` پیش‌فرض کد جدید و KMP است.
- برای Room از KSP، DAOهای `suspend` و `Flow` و schemaهای ثبت‌شده در git استفاده کنید.
- `@Relation` را با `@Transaction` بنویسید؛ relation کلید خارجی نیست.
- نبودن migration یعنی crash، نه پاک شدن داده. تست از هر نسخهٔ منتشرشده.
- queryهای `Flow` با هر نوشتن روی جدول‌هایشان دوباره اجرا می‌شوند؛ پس `distinctUntilChanged()`.
- DataStore برای تنظیمات کوچک؛ Paging 3 همراه با `cachedIn(viewModelScope)` برای لیست‌های بزرگ.
- ViewModel، Room، DataStore و هستهٔ Paging در کد مشترک KMP اجرا می‌شوند.

روز ۷ در راه است: Dependency Injection. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700